PROGRAMACIÓN EN ESP32 LENGUAJE C/C++

- Creado por **Dennis Ritchie** en los laboratorios de AT&T entre 1969 y 1973
- Para escribir el SO Unix en un lenguaje de alto nivel e independiente del hardware
- Basado en el B de Ken Thompson
- En 1978 Ritchie junto a **Bryan Kernigan** publican el libro: “el lenguaje de programación C”
- A partir de dicho libro surge el lenguaje conocido como “el C de Kernigan y Ritchie” o simplemente “K&R C”
- A mediados de los 80 **Bjarne Stroustrup** crea el lenguaje C++

FUNDAMENTOS

- Lenguaje de programación estructurada + POO.
- Originalmente concebido como de alto nivel, hoy de medio/bajo nivel en comparación con los lenguajes actuales.
- Lenguaje Compilado, es decir que se parte del código fuente se compila en una plataforma y se obtiene un ejecutable para esa plataforma.
- Manejo libre y a cargo del programador de la memoria.
- Lenguaje fuertemente tipado.

SINTAXIS

- ; separa comandos sucesivos
- { … } agrupan comandos dentro de una estructura lógica, útiles para mantener balanceado el código.

Dos posibilidades para definir comentarios en el código de un sketch:

- Comentario de una sola línea: //comentario...
- Comentario en más de una línea : /*...*/

TIPOS DE DATOS

- **bool** (8 bit)- lógico simple verdadero/falso.
- **byte** (8 bit)- número sin signo entre 0 y 255.
- **char** (8 bit)- número con signo, entre -128 y 127. En algunos casos el compilador intentará interpretar este tipo de dato como un caracter, lo que puede generar resultados inesperados.
- **unsigned char** (8 bit)- lo mismo que ‘byte’; si es que eso es lo que se necesita, se recomienda usar ‘byte’, para que el código sea más claro.
- **word** (16 bit)- número sin signo entre, 0 y 65535.
- **unsigned int** (16 bit)- lo mismo que ‘word’. Utilizar ‘word’ por simplicidad y brevedad.
- **int** (16 bit)- número con signo, entre -32768 y 32767. Este tipo es el más usado para variables de propósito general en Arduino, en los códigos de ejemplo que vienen con el IDE.

TIPOS DE DATOS

- **unsigned long** (32 bit)- número sin signo entre 0 y 4.294.967.295.

Este tipo se usa comúnmente para almacenar el resultado de la función millis(), la cual retorna el tiempo que el código actual ha estado corriendo, en milisegundos.

- **short** (16 bit)- número con signo, entre -32.768 y 32.767
- **long** (32 bit)- número con signo, entre -2.147.483.648 y 2.147.483.647
- **float** (32 bit)- número con signo, entre -3,4028235E38 y 3,4028235E38

El Punto Flotante no es un tipo nativo en ESP32; el compilador debe realizar varios saltos para poder hacerlo funcionar. Evitarlo siempre que sea posible.

DEFINICIÓN DE VARIABLES

Global

```
int globalVariable = 10; // variable global
void setup() {
  // código
}
void loop() {
  // código que usa la variable global
}
```

Local

```
void setup() {
  int localVariable = 5; // variable local
  // código
}
void loop() {
  // código que no puede acceder a la variable local
}
```

Constante

```
const int MAX_VALUE = 100; // variable constante
void setup() {
  // código que no puede modificar la variable constante
}
void loop() {
  // código que no puede modificar la variable constante
}
```

DEFINICIÓN DE VARIABLES

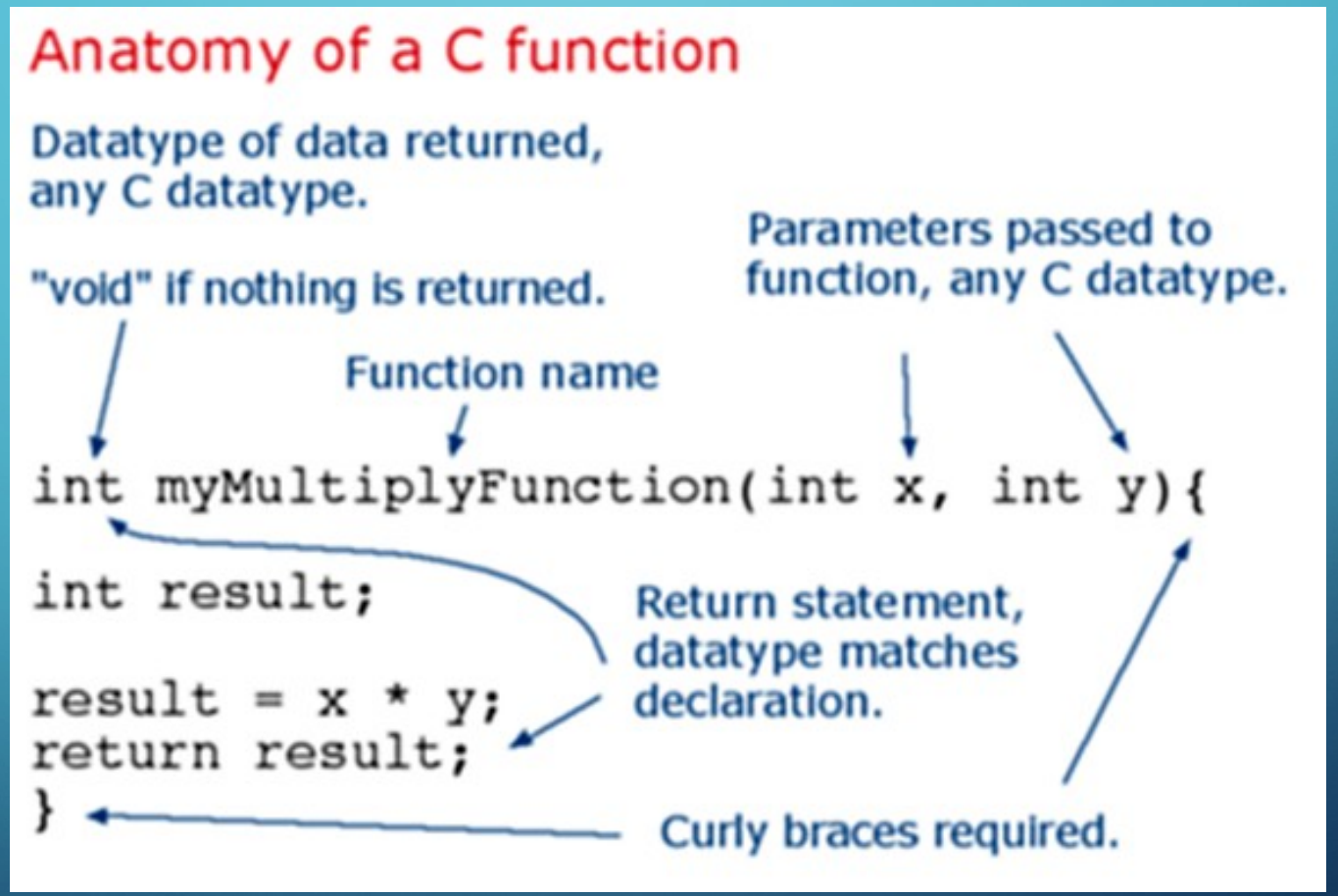
Estática

```
void setup() {
  // código que no puede cambiar el valor de la variable estática
}
void loop() {
  static int contador = 0; // variable estática inicializada
  contador++;
  // código que usa la variable estática
}
```

Volátil

```
volatile int sensorValue; // variable volátil
void setup() {
  // código que puede cambiar el valor de la variable volátil
}
void loop() {
  // código que puede leer el valor actual de la variable volátil
}
```

USO DE FUNCIONES



PARÁMETROS POR VALOR

Se crea una variable LOCAL con el valor que tiene el parámetro.

Dicha variable (y su valor) se pierde cuando finaliza la ejecución del código de la función.

```
void setup(){
  Serial.begin(9600);
}

void loop() {
  int i = 2;
  int j = 3;
  int k;

  k = myMultiplyFunction(i, j); // k now contains 6
  Serial.println(k);
  delay(500);
}

int myMultiplyFunction(int x, int y){
  int result;
  result = x * y;
  return result;
}
```

PARÁMETROS POR REFERENCIA

No crea una variable sino un **puntero** a la posición de memoria de la variable.

Ese valor de la variable modificada permanece al finalizar la ejecución del código de la función.

```
void incrementar(int valor, int &resultado) {
    resultado = valor + 1;
}

int main() {
    int numero = 5;
    int resultado;

    incrementar(numero, resultado);

    // resultado ahora es igual a 6
    return 0;
}
```

PARÁMETROS POR REFERENCIA

Punteros

```
void function() {
    int x = 5;
    int* ptr = &x;  // Puntero que apunta a la dirección de memoria de x
    *ptr = 10;  // Modificar el valor de x a través del puntero
}
```

- Los punteros son variables que almacenan direcciones de memoria.
- Puedes utilizar punteros para acceder y manipular datos en la memoria.
- Puedes asignar memoria dinámicamente, acceder a elementos de arreglos y estructuras, y trabajar con estructuras de datos complejas

PUNTEROS:

ACCESO Y MANIPULACIÓN DE LA MEMORIA

Los **punteros** permiten acceder y manipular directamente la memoria de un programa.

Esto brinda flexibilidad para trabajar con:

- Estructuras de datos complejas
- Arreglos dinámicos
- Listas enlazadas Punteros
- Estructuras personalizadas.

PUNTEROS:

EFICIENCIA Y AHORRO DE MEMORIA

El uso adecuado de punteros puede ayudar a optimizar la eficiencia del código y **ahorrar memoria**.

En lugar de copiar grandes bloques de datos, puedes pasar punteros a esos datos, evitando así la duplicación innecesaria de memoria.

CONSTANTES Y FUNCIONES BASICAS DE E/S

Constantes más usadas	
`INPUT/OUTPUT`	Permite definir entradas o salidas mendiante la función pinMode().
`HIGH (3.3V)/LOW (0V)`	Permiten indicar un valor Alto o Bajo de tensión dependiendo la configuración del pin.
`LED_BUILTIN`	ESP32 tiene un pin conectado a un LED integrado en serie con una resistencia. La constante LED_BUILTIN es el número del pin al que está conectado el LED integrado. Este valor es 2.

CONSTANTES Y FUNCIONES BASICAS DE E/S

E/S Digitales	
`pinMode(pin, modo)`	Configura el pin especificado para que se comporte como una salida (OUTPUT), o una entrada (INPUT) o bien INPUT_PULLUP, INPUT_PULLDOWN para entradas con resistencias integradas.
`digitalWrite(pin, valor)`	Escribe un valor HIGH o un LOW en un pin digital.
`digitalRead(pin)`	Lee el valor de un pin digital especificado, ya sea HIGH o LOW.

CONSTANTES Y FUNCIONES BASICAS DE E/S

E/S Analógicas	
`analogRead(pin)`	Lee el valor del pin analógico especificado. (12 bits → 0 a 4095)
`analogWrite(pin,valor)`	Escribe un valor analógico (onda PWM) en un pin. Se puede usar para encender un LED con diferentes brillos o impulsar un motor a varias velocidades.
`analogReference()`	Configura el voltaje de referencia utilizado para la entrada analógica (es decir, el valor utilizado como la parte superior del rango de entrada). En Esp32 el valor por defecto es 3.3V

CONSTANTES Y FUNCIONES BASICAS DE E/S

E/S Analógicas	
`analogReadResolution(bits)`	Setea el tamaño en bits del valor leído por analogRead(). Por defecto es 12 bits.
`analogWriteResolution(bits)`	Setea el tamaño en bits del valor de salida de analogWrite(). Por defecto es 8 bits.

CONSTANTES Y FUNCIONES BASICAS DE E/S

Comunicación Serie	
`Serial.begin(velocidad)`	Establece la velocidad de datos en bits por segundo (baudios) para la transmisión de datos en serie. Un segundo argumento opcional configura los bits de datos (Serial.begin(veloc, config)), paridad y parada. El valor predeterminado es 8 bits de datos, sin paridad, un bit de parada.
`Serial.read()`	Lee los datos seriales entrantes.
`Serial.print(val, [format])`	Imprime datos en el puerto serial como texto ASCII legible por humanos. Este comando puede tomar muchas formas. Los números se imprimen utilizando un carácter ASCII para cada dígito. Los flotantes se imprimen de manera similar como dígitos ASCII, por defecto con dos decimales. Los bytes se envían como un solo carácter. Los caracteres y las cadenas se envían tal cual.

CONSTANTES Y FUNCIONES BASICAS DE E/S

Comunicación Serie	
`Serial.println(val, [format])`	Imprime datos en el puerto serie como texto ASCII legible por humanos seguido de un carácter de retorno de carro (ASCII 13 o '\r') y un carácter de nueva línea (ASCII 10 o '\n'). Este comando toma las mismas formas que Serial.print() .
`Serial.available()`	Obtiene la cantidad de bytes (caracteres) disponibles para leer desde el puerto serie. Estos son datos que ya llegaron y se almacenaron en el búfer de recepción en serie (que contiene 64 bytes).

https://www.arduino.cc/reference/en/